EF Core Best Practices and Architecture (Exercise)

1. Introduction

Welcome back!

In the previous exercise, we built the initial structure of our ASP.NET Core application and successfully executed the **JSON import** functionality. Today we **continue from that point**, but we shift our focus to how *real, maintainable, professional* applications are built.

This lecture is divided into three key themes:

1. **Finishing the Import Module** – Implementing the XML Import
2. **Architecture & Clean Code** – Moving logic from Controllers into Services
3. **Validation & Debugging** – Understanding broken data and catching errors properly

By the end of this lecture, you will clearly understand *why architecture matters* and *how to write cleaner, safer, and more scalable applications*

2. Completing the Import: XML

In the last session, we implemented the **JSON import** live.
Today, your task is to finish the **XML import**, using the exact same structure and concepts.

Your goals:

- **Read** XML input
- **Deserialize** into Import DTOs
- **Validate** each DTO
- **Map** DTO → Entity
- **Save** valid data into the database

What matters here:

✓ Data must pass validation before we save it.
✓ Incorrect records must be skipped.
✓ XML structure must match our DTO structure exactly.

Tip: XML is less forgiving than JSON. Expect more real-world issues, and use breakpoints often

3. Clean Architecture: Moving Logic into Services

Last time, our controllers became **"fat"**

They were **doing too much**: validation, mapping, database queries, decisions.
This is not how modern ASP.NET applications are built.

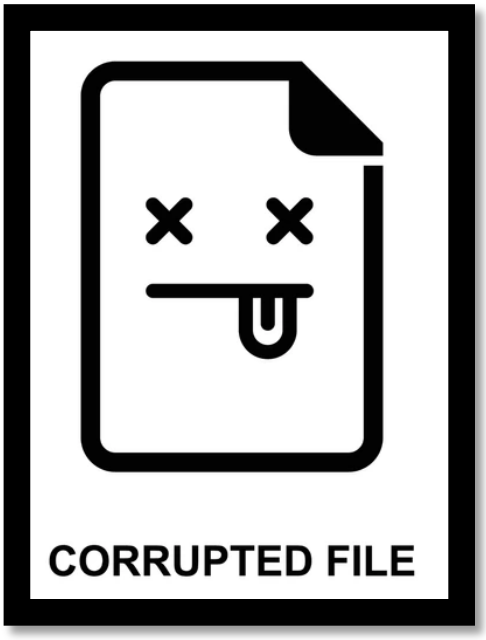
Today's principle:

Controllers should **only take the request and return the response**. Nothing more.

All logic belongs in **Services**.

4. Validation: Understanding Broken Data

In real applications, the **worst errors come from bad input**.
That's why today we work with **broken data on purpose**.



Types of validation you will see today:



Data Annotation Validation

[Required], [Range], [StringLength], etc.

Custom Business Validation in Services

Example: "A cinema cannot have two movies with the same name."

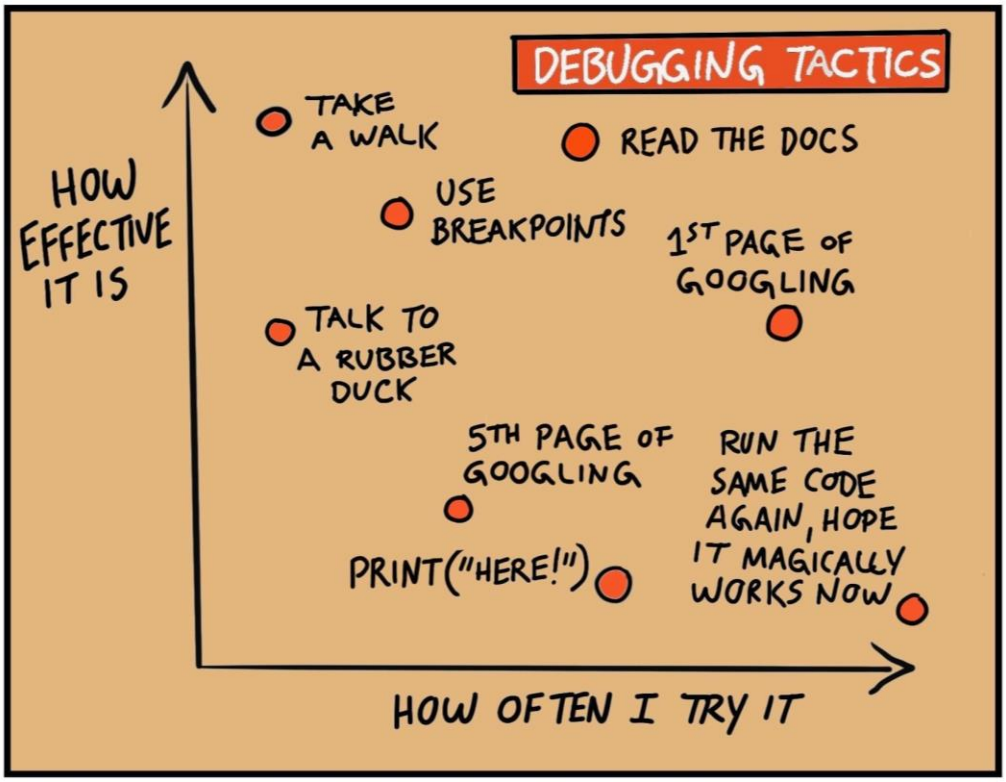
Import Validation (XML/JSON)

Strict checking of DTOs before mapping.

Database-Level Validation

Foreign keys, unique indexes, constraints.

Debugging Validation



We will **learn how to catch validation errors** using:

- Breakpoints
- ModelState.Errors
- Logging
- Checking DTO values before mapping
- Understanding XML deserialization errors

Your debugging checklist:

✓ Did the **DTO match the XML exactly**?
✓ Did you **decorate all required properties**?
✓ Is the **relationship valid**?
✓ Are you trying to **insert duplicates**?
✓ Are **identifiers correct**?

By intentionally **breaking data**, you will understand **why validation exists** and **how to track failures** when they happen.